Name: P ARCHITH Roll Number: 2303A52258 Batch: 45

Assignment Number:12.3(Present assignment number)/**24**(Total number of assignments)

Lab 12: Algorithms with AI Assistance Sorting, Searching, and Algorithm Optimization Using AI Tools

Lab Objectives

The objectives of this laboratory exercise are to:

- Apply AI-assisted programming techniques to implement sorting and searching algorithms.
- Analyze and compare algorithm efficiency using time and space complexity.
- Understand how AI tools can suggest optimizations and alternative algorithmic approaches.
- Strengthen problem-solving skills through real-world, data-driven scenarios.

Learning Outcomes

After completing this lab, students will be able to:

- Implement and optimize classic algorithms using AI-assisted coding tools.
- Compare multiple algorithms for the same problem and justify their selection.
- Measure and analyze runtime performance using experimental data.
- Critically review and refine AI-generated algorithmic solutions.

Task 1: Sorting Student Records for Placement Drive

Scenario

SR University's Training and Placement Cell needs to shortlist candidates efficiently during campus placements. Student records must be sorted by **CGPA in descending order**.

Tasks

1. Use **GitHub Copilot** to generate a program that stores student records (Name, Roll Number, CGPA).
2. Implement the following sorting algorithms using AI assistance:
    - Quick Sort
    - Merge Sort
3. Measure and compare runtime performance for large datasets.
4. Write a function to display the **top 10 students** based on CGPA.

Expected Outcome

- Correctly sorted student records.

- Performance comparison between Quick Sort and Merge Sort.
- Clear output of top-performing students.

Prompt: Create a Python program to manage student records for a university placement drive.

Requirements:

1. Each student record should contain:

   - Name

   - Roll Number

   - CGPA

2. Store multiple student records in a list.

3. Implement two sorting algorithms to sort students by CGPA in descending order:

   - Quick Sort

   - Merge Sort

4. Generate a large dataset (for example 10,000 students with random CGPA values between 5.0 and 10.0) to test performance.

5. Measure runtime performance of both sorting algorithms using the time module.

6. After sorting, display the top 10 students based on CGPA.

7. Output should include:

   - Time taken by Quick Sort

   - Time taken by Merge Sort

   - Top 10 students with Name, Roll Number, and CGPA

Use clean Python code with functions for:

- generating students

- quick sort

- merge sort

- displaying top students

Code:

```python
import random
import time
# Student class
class Student:
    def __init__(self, name, roll, cgpa):
        self.name = name
        self.roll = roll
        self.cgpa = cgpa
    def __repr__(self):
        return f"{self.name} | {self.roll} | CGPA: {self.cgpa:.2f}"
# Generate random students
def generate_students(n):
    students = []
    for i in range(n):
        name = "Student_" + str(i + 1)
        roll = "SRU" + str(1000 + i)
        cgpa = round(random.uniform(5.0, 10.0), 2)
        students.append(Student(name, roll, cgpa))
    return students
# Quick Sort
def quick_sort(arr):
    if len(arr) <= 1:
        return arr
```

Output:

```
PS D:\3-2\AI Assitant Coding\Code Files> & "d:\3-2\AI Assitant Coding\(
(.venv) PS D:\3-2\AI Assitant Coding\Code Files>  d:; cd 'd:\3-2\AI As
\.venv\Scripts\python.exe' 'c:\Users\SNEHITH\.vscode\extensions\ms-pyt
332' '--' 'D:\3-2\AI Assitant Coding\Code Files\Task-12.3.py'
Runtime Performance Comparison

Quick Sort Time : 0.018899202346801758
Merge Sort Time : 0.03379940986633301

Top 10 Students Based on CGPA

Name | Roll No | CGPA
-----------------------------
Student_552 | SRU1551 | CGPA: 10.00
Student_1907 | SRU2906 | CGPA: 10.00
Student_2898 | SRU3897 | CGPA: 10.00
Student_3077 | SRU4076 | CGPA: 10.00
Student_4271 | SRU5270 | CGPA: 10.00
Student_4493 | SRU5492 | CGPA: 10.00
Student_6458 | SRU7457 | CGPA: 10.00
Student_6580 | SRU7579 | CGPA: 10.00
Student_7295 | SRU8294 | CGPA: 10.00
Student_8970 | SRU9969 | CGPA: 10.00
(.venv) PS D:\3-2\AI Assitant Coding\Code Files>
```

Task 2: Implementing Bubble Sort with AI Comments

- **Task**: Write a Python implementation of **Bubble Sort**.
- **Instructions**:
    - Students implement Bubble Sort normally.
    - Ask AI to generate **inline comments explaining key logic** (like swapping, passes, and termination).
    - Request AI to provide **time complexity analysis**.
- **Expected Output**:
    - A Bubble Sort implementation with AI-generated explanatory comments and complexity analysis.

Prompt: Write a Python program that implements the Bubble Sort algorithm.

Requirements:

1. The program should sort a list of numbers in ascending order.

2. Add clear inline comments explaining:

  - Each pass of the algorithm

  - The comparison between elements

  - The swapping of elements

  - The stopping condition when no swaps occur

3. After sorting, print the sorted list.

Also include a short explanation of the time complexity of Bubble Sort in:

- Best case

- Average case

- Worst case

Code:

```python
# Bubble Sort Implementation with Detailed Comments
def bubble_sort(arr):
    n = len(arr)
    # Outer loop represents the number of passes
    # After each pass, the largest unsorted element "bubbles up"
    # to its correct position at the end of the list
    for i in range(n):
        # This flag checks if any swap happens during a pass
        # If no swaps occur, the list is already sorted
        swapped = False
        # Inner loop performs pairwise comparisons
        # n-i-1 because the last i elements are already sorted
        for j in range(0, n - i - 1):
            # Compare adjacent elements
            if arr[j] > arr[j + 1]:
                # Swap elements if they are in the wrong order
                arr[j], arr[j + 1] = arr[j + 1], arr[j]
                # Mark that a swap has occurred
                swapped = True
        # If no swapping happened in this pass,
        # the array is already sorted so we stop early
        if not swapped:
            break
    return arr
```

Output:

```
(.venv) PS D:\3-2\AI Assitant Coding\Code Files>  d:; cd 'd:\3-2\AI Assitant Coding\(
\.venv\Scripts\python.exe' 'c:\Users\SNEHITH\.vscode\extensions\ms-python.debugpy-202
902' '--' 'D:\3-2\AI Assitant Coding\Code Files\Task-12.3.py'
Original List: [64, 34, 25, 12, 22, 11, 90]
Sorted List: [11, 12, 22, 25, 34, 64, 90]
(.venv) PS D:\3-2\AI Assitant Coding\Code Files>
```

Task 3: Quick Sort and Merge Sort Comparison

- **Task:** Implement Quick Sort and Merge Sort using recursion**.**
- **Instructions:**
    - Provide AI with partially completed functions for recursion.
    - Ask AI to complete the missing logic and add docstrings.
    - Compare both algorithms on random, sorted, and reverse-sorted lists.
- **Expected Output:**
    - Working Quick Sort and Merge Sort implementations.
    - AI-generated explanation of average, best, and worst-case complexities.

Prompt: Complete a Python program that implements Quick Sort and Merge Sort using recursion.

Requirements:

1. Provide recursive implementations of:
   - Quick Sort
   - Merge Sort
2. Add proper Python docstrings to each function explaining:
   - What the function does
   - Parameters
   - Return value
3. Generate three types of datasets:
   - Random list
   - Already sorted list
   - Reverse sorted list
4. Run both algorithms on each dataset and measure execution time using the time module.
5. Print a comparison table showing:
   - Algorithm name
   - Dataset type
   - Execution time
6. At the end include a short explanation of:
   - Best case complexity
   - Average case complexity
   - Worst case complexity

for both Quick Sort and Merge Sort.

Code:

```python
import random
import time
def quick_sort(arr):
    """
    Quick Sort using recursion.

    Parameters:
    arr (list): List of numbers to be sorted

    Returns:
    list: Sorted list in ascending order
    """
    # Base case: if list has 0 or 1 element it is already sorted
    if len(arr) <= 1:
        return arr
    # Choose pivot element
    pivot = arr[len(arr) // 2]
    # Partition the list into three parts
    left = [x for x in arr if x < pivot]
    middle = [x for x in arr if x == pivot]
    right = [x for x in arr if x > pivot]
    # Recursively sort left and right parts
    return quick_sort(left) + middle + quick_sort(right)
def merge_sort(arr):
    """
```

Output:

```
(.venv) PS D:\3-2\AI Assitant Coding\Code Files>  d:; cd 'd:\3-2\AI Assitant Coding\
\.venv\Scripts\python.exe' 'c:\Users\SNEHITH\.vscode\extensions\ms-python.debugpy-20
149' '--' 'D:\3-2\AI Assitant Coding\Code Files\Task-12.3.py'

Dataset: Random List
Quick Sort Time : 0.023772001266479492
Merge Sort Time : 0.03007197380065918

Dataset: Sorted List
Quick Sort Time : 0.0160830020904541
Merge Sort Time : 0.02062845230102539

Dataset: Reverse Sorted List
Quick Sort Time : 0.016997814178466797
Merge Sort Time : 0.02090620994567871
(.venv) PS D:\3-2\AI Assitant Coding\Code Files>
```

Task 4 (Real-Time Application – Inventory Management System)

Scenario: A retail store's inventory system contains thousands of products, each with attributes like product ID, name, price, and stock quantity. Store staff need to:

1. **Quickly search for a product by ID or name.**
2. **Sort products by price or quantity for stock analysis.**

Task:

- **Use AI to suggest the most efficient search and sort algorithms for this use case.**
- **Implement the recommended algorithms in Python.**
- **Justify the choice based on dataset size, update frequency, and performance requirements.**

Expected Output:

- **A table mapping operation → recommended algorithm → justification.**
- **Working Python functions for searching and sorting the inventory.**

Prompt: Create a Python program for a retail store Inventory Management System.

Requirements:

1. Each product should have:

- Product ID

- Product Name

- Price

- Stock Quantity

2. Implement efficient algorithms for:

- Searching product by ID

- Searching product by Name

- Sorting products by Price

- Sorting products by Quantity

3. Use the following algorithms:

- Binary Search for searching by Product ID

- Linear Search for searching by Product Name

- Quick Sort for sorting by Price

- Merge Sort for sorting by Quantity

4. Generate a sample inventory dataset with multiple products.

5. Print results for:

- **Searching a product**

- **Sorted list of products by price**

- **Sorted list of products by quantity**

6. Write clean Python functions with comments explaining each step.

Code:

```python
# Product class
class Product:
    def __init__(self, pid, name, price, quantity):
        self.pid = pid
        self.name = name
        self.price = price
        self.quantity = quantity
    def __repr__(self):
        return f"{self.pid} | {self.name} | Price:{self.price} | Qty:{self.quantity}"
# Binary Search (Search by Product ID)
def binary_search(products, target_id):
    low = 0
    high = len(products) - 1
    while low <= high:
        mid = (low + high) // 2
        if products[mid].pid == target_id:
            return products[mid]
        elif products[mid].pid < target_id:
            low = mid + 1
        else:
            high = mid - 1
    return None
# Linear Search (Search by Name)
def linear_search(products, name):
    for product in products:
```

Output:

```
(.venv) PS D:\3-2\AI Assitant Coding\Code Files>  d:; cd 'd:\3-2\AI Assitant Coding\
\.venv\Scripts\python.exe' 'c:\Users\SNEHITH\.vscode\extensions\ms-python.debugpy-20
196' '--' 'D:\3-2\AI Assitant Coding\Code Files\Task-12.3.py'
Search by Product ID:
103 | Keyboard | Price:1200 | Qty:80

Search by Product Name:
104 | Monitor | Price:12000 | Qty:40

Products Sorted by Price:
102 | Mouse | Price:500 | Qty:150
103 | Keyboard | Price:1200 | Qty:80
105 | Printer | Price:8000 | Qty:20
104 | Monitor | Price:12000 | Qty:40
101 | Laptop | Price:75000 | Qty:15

Products Sorted by Quantity:
101 | Laptop | Price:75000 | Qty:15
105 | Printer | Price:8000 | Qty:20
104 | Monitor | Price:12000 | Qty:40
103 | Keyboard | Price:1200 | Qty:80
102 | Mouse | Price:500 | Qty:150
(.venv) PS D:\3-2\AI Assitant Coding\Code Files>
```

Task 5: Real-Time Stock Data Sorting & Searching

Scenario:
An AI-powered FinTech Lab at SR University is building a tool for analyzing stock price movements. The requirement is to quickly sort stocks by daily gain/loss and search for specific stock symbols efficiently.

- **Use GitHub Copilot to fetch or simulate stock price data (Stock Symbol, Opening Price, Closing Price).**
- **Implement sorting algorithms to rank stocks by percentage change.**
- **Implement a search function that retrieves stock data instantly when a stock symbol is entered.**
- **Optimize sorting with Heap Sort and searching with Hash Maps.**
- **Compare performance with standard library functions (sorted(), dict lookups) and analyze trade-offs.**

Prompt: Create a Python program that simulates a stock market data analyzer.

Requirements:

1. Each stock record should contain:

- Stock Symbol

- Opening Price

- Closing Price

- Percentage Change

2. Generate a dataset of stocks (at least 20–50 stocks) with random opening and closing prices.

3. Implement:

- Heap Sort to sort stocks by percentage change (highest gain first).

- Hash Map (dictionary) to search stocks instantly using stock symbol.

4. Also compare performance with:

- Python sorted() function

- Python dictionary lookup

5. Print:

- Sorted stock rankings by percentage change

- Result of searching a stock symbol

- Execution time comparison

6. Use clean functions and comments explaining logic.

Code:

```python
import random
import time
# Stock class
class Stock:
    def __init__(self, symbol, open_price, close_price):
        self.symbol = symbol
        self.open_price = open_price
        self.close_price = close_price
        self.change = ((close_price - open_price) / open_price) * 100
    def __repr__(self):
        return f"{self.symbol} | Open:{self.open_price:.2f} Close:{self.close_price:.2f} Change:{self.change:.2f}
# Generate simulated stock data
def generate_stocks(n):
    symbols = [f"STK{i}" for i in range(n)]
    stocks = []
    for s in symbols:
        open_price = random.uniform(100, 500)
        close_price = random.uniform(100, 500)
        stocks.append(Stock(s, open_price, close_price))
    return stocks
# Heapify function for Heap Sort
def heapify(arr, n, i):
    largest = i
    left = 2*i + 1
    right = 2*i + 2
```

Output:

```
(.venv) PS D:\3-2\AI Assitant Coding\Code Files>  d:; cd 'd:\3-2\AI Assitant Coding\Code Files';
\.venv\Scripts\python.exe' 'c:\Users\SNEHITH\.vscode\extensions\ms-python.debugpy-2025.18.0-win32
122' '--' 'D:\3-2\AI Assitant Coding\Code Files\Task-12.3.py'
Top 10 Stocks by Percentage Gain/Loss

STK12 | Open:103.17 Close:406.76 Change:294.27%
STK13 | Open:145.29 Close:498.34 Change:243.00%
STK0 | Open:107.96 Close:329.07 Change:204.80%
STK23 | Open:150.09 Close:452.69 Change:201.60%
STK10 | Open:179.45 Close:340.57 Change:89.78%
STK21 | Open:134.75 Close:244.11 Change:81.16%
STK15 | Open:130.46 Close:224.36 Change:71.98%
STK1 | Open:295.89 Close:483.28 Change:63.33%
STK6 | Open:297.04 Close:484.07 Change:62.97%
STK2 | Open:231.15 Close:358.04 Change:54.90%

Search Stock:
STK10 | Open:179.45 Close:340.57 Change:89.78%

Performance Comparison
Heap Sort Time: 5.221366882324219e-05
Python sorted() Time: 7.152557373046875e-06
Hash Map Search Time: O(1) average
(.venv) PS D:\3-2\AI Assitant Coding\Code Files>
```